

QuantL

Conception d'un mini-langage de programmation quantique avec compilateur (Flex/Bison/C)

NGUEUDJANG DJOMO ALAIN GILDAS – 22W2183

Université de Yaoundé I
Faculté des Sciences
Département d'Informatique — Master 1

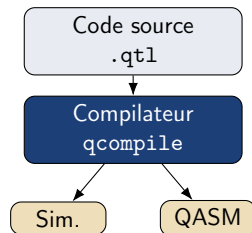
12 juillet 2026

Superviseur: Dr. THOMAS MESSI NGUELE

- 1 Introduction et motivation
- 2 Architecture globale du projet
- 3 Le langage QuantL
- 4 Analyse lexicale et syntaxique
- 5 Arbre syntaxique abstrait (AST)
- 6 Analyse sémantique
- 7 Représentation intermédiaire (IR)
- 8 Moteur de simulation quantique
- 9 Génération de code OpenQASM 2.0
- 10 Étude de cas : trois circuits de référence
- 11 Interface graphique (PyQt6)
- 12 Compilation, tests et validation
- 13 Bilan, limites et perspectives

Contexte général

- L'informatique quantique impose de nouveaux paradigmes de programmation : **superposition**, **intrication**, **mesure probabiliste**.
- Les langages existants (Qiskit, Cirq, Q#, OpenQASM) sont soit des bibliothèques hôtes (Python), soit des DSL bas niveau.
- **QuantL** propose un langage *autonome*, compilé, avec :
 - une syntaxe impérative simple et lisible ;
 - une chaîne de compilation classique (lexer/parser/AST) ;
 - un moteur de simulation à vecteur d'état ;
 - un export vers le format standard **OpenQASM 2.0** ;
 - une interface graphique complète (PyQt6).



Objectif général

Concevoir un langage de programmation quantique compilé, complet de bout en bout : de la spécification lexicale jusqu'à la simulation numérique et l'export interopérable.

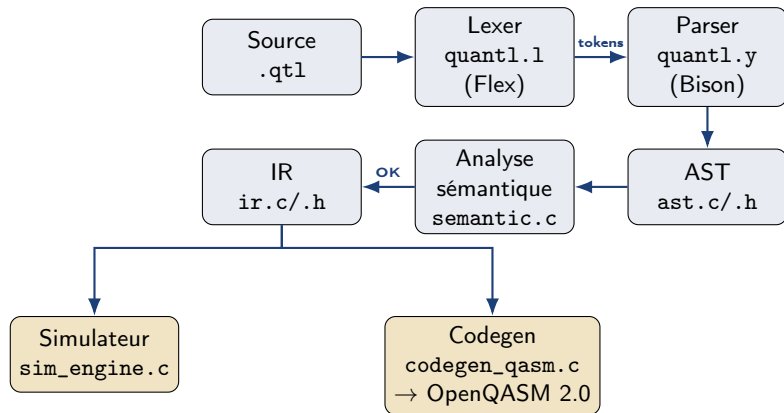
Objectifs techniques

- Définir une grammaire formelle (EBNF/Bison) pour les circuits quantiques
- Construire un AST typé représentant fidèlement le programme
- Implémenter une analyse sémantique robuste (portées, types, usages)
- Générer une représentation intermédiaire (IR) linéaire et indépendante du back-end

Objectifs fonctionnels

- Simuler l'évolution d'un état quantique par calcul matriciel sur amplitudes complexes
- Produire du code **OpenQASM 2.0** interopérable avec les SDK existants
- Offrir une interface graphique pédagogique (édition, circuit, simulation, export)

Vue d'ensemble — pipeline de compilation



Pipeline strictement séquentiel piloté par `main.c` (options `-source`, `-mode`, `-seed`, `-shots`).

- Syntaxe **impérative** proche du C/OpenQASM : familière, peu de sucre syntaxique.
- Deux espaces de registres explicitement typés :
 - qreg — registre **quantique** (qubits, état en superposition)
 - creg — registre **classique** (bits résultant d'une mesure)
- Toute opération référence un registre indicé : `q[0]`, `c[1]`, ...
- Structures de contrôle minimales mais suffisantes pour l'enseignement : `if`, `repeat`, `barrier`.
- Objectif pédagogique assumé : chaque construction du langage correspond à une primitive standard de circuit quantique.

Syntaxe — déclarations et portes à un qubit

```
// Déclarations de registres
qreg q[3];      // 3 qubits
creg c[3];      // 3 bits classiques

// Portes a un qubit (sans angle)
h q[0];         // Hadamard
x q[1];         // Pauli-X (NOT)
y q[1];         // Pauli-Y
z q[2];         // Pauli-Z
s q[0];         // Phase S
t q[0];         // Phase T (pi/4)

// Portes de rotation (avec angle)
rx q[0](pi/2);
ry q[1](1.5708);
rz q[2](pi);
```

Table des mots-clés porte 1-qubit

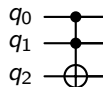
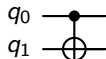
Token	Portes
PORTE1Q	h, x, y, z, s, t
PORTEROT	rx, ry, rz

Le lexer distingue les portes *sans angle* des portes de *rotation*, qui exigent un paramètre réel entre parenthèses (angle : FLOTTANT | ENTIER).

Syntaxe — portes multi-qubits

```
// Portes a deux qubits
cx q[0], q[1];      // CNOT (controle,
                   // cible)
cz q[0], q[2];      // Controlled-Z
swap q[1], q[2];     // Echange

// Porte a trois qubits
toffoli q[0], q[1], q[2]; // CCNOT
```



Syntaxe — mesure, contrôle conditionnel, synchronisation

```
// Mesure d'un qubit vers un bit classique
measure q[0] -> c[0];

// Instruction conditionnelle sur un bit classique mesure
if (c[0] == 1) x q[1];

// Barriere de synchronisation (empeche certaines optimisations)
barrier q[0], q[1], q[2];

// Repetition d'un bloc d'instructions
repeat (5) {
    h q[0];
    measure q[0] -> c[0];
}

// Affichage de diagnostics
print state;    // vecteur d'etat complet
print probs;    // distribution de probabilites + mesures
```

```
programme ::= instruction*  
instruction ::= decl_qreg | decl_creg | instr_porte | instr_mesure  
              | instr_if | instr_print | instr_barrier | instr_repeat  
decl_qreg  ::= qreg IDENT [ ENTIER ] ;  
instr_porte ::= PORTE1Q IDENT[ENTIER] ((angle))? ;  
              | PORTE2Q IDENT[ENTIER], IDENT[ENTIER] ;  
              | toffoli IDENT[ENTIER], IDENT[ENTIER], IDENT[ENTIER] ;  
instr_if   ::= if (IDENT[ENTIER] == ENTIER) instruction  
instr_repeat ::= repeat (ENTIER) { instruction* }  
angle      ::= FLOTTANT | ENTIER
```

Remarque

La grammaire est implémentée directement en **Bison** ([quantl.y](https://www.gnu.org/software/bison/)) avec `%define parse.error verbose` pour des messages d'erreur syntaxique explicites, et gérée sans conflits shift/reduce résiduels (`-v` génère `quantl.output`).

```
"qreg" { return QREG; }
"creg" { return CREG; }
"h"|"x"|"y"|"z"|"s"|"t"
    { yylval.str = strdup(yytext);
      return PORTE1Q; }
"rx"|"ry"|"rz"
    { yylval.str = strdup(yytext);
      return PORTEROT; }
"cx"|"cz"|"swap"
    { yylval.str = strdup(yytext);
      return PORTE2Q; }
"pi"   { yylval.dval = 3.14159265358979;
      return FLOTTANT; }
[a-zA-Z_][a-zA-Z0-9_]*
    { yylval.str = strdup(yytext);
      return IDENT; }
```

Caractéristiques

- Reconnaissance des mots-clés *avant* la règle générique IDENT (priorité à la règle la plus longue/la plus spécifique de Flex)
- Gestion du comptage de lignes (`ligne_courante`) pour les diagnostics d'erreur
- Ignore les commentaires `//...` et les espaces/tabulations
- Rejette proprement les caractères invalides avec message d'erreur explicite (protection contre BOM/Unicode invisibles)
- Constante `pi` injectée directement comme littéral flottant au niveau lexical

Analyseur syntaxique (Bison) — règle `instr_if`

```
instr_if:
  IF '(', IDENT '[', ENTIER ']', EGALegal ENTIER ')',
  instruction {
    $$ = ast_if($3, $5, $8, $10, ligne_courante);
  }
;
```

- La règle `instr_if` est **récursive à droite** : le corps du `if` est lui-même une *instruction* quelconque (porte, mesure, print, etc.), pas un bloc obligatoire.
- Les valeurs sémantiques (`$3`, `$5`, ...) sont combinées via `ast_if(...)` pour construire un nœud `N_IF` homogène.
- L'union Bison `%union` porte quatre types de valeurs : `ival`, `dval`, `str`, `noeud` — reflet direct des types de tokens (`ENTIER`, `FLOTTANT`, `IDENT`, non-terminaux).
- La règle `liste_ref_qubits` (pour `barrier`) illustre une construction incrémentale par `realloc` d'un tableau de `Reference`.

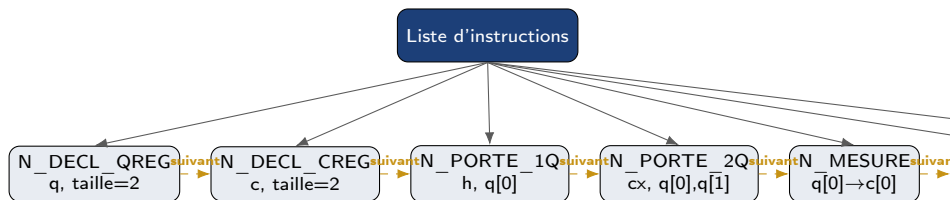
Structure de données de l'AST

```
typedef struct NoeudAST {
    TypeNoeud type;
    int ligne;
    struct NoeudAST* suivant;
    union {
        struct { char* nom_registre;
                  int taille;
                  TypeRegistre type_reg;
                  } decl;
        struct { char* nom_porte;
                  char* nom_reg;
                  int indice_qubit;
                  double angle;
                  bool a_angle; }
                  porte_1q;
        struct { ... } porte_2q;
        struct { ... } porte_toffoli;
        struct { ... } mesure;
        struct { ... } if_instr;
        struct { ... } print_instr;
        struct { ... } barrier;
        struct { ... } repeat;
    };
} NoeudAST;
```

Points clés de conception

- Un **unique type de nœud** (NoeudAST) avec une union discriminée par le champ type (TypeNoeud)
- Chaînage suivant → liste simplement chaînée d'instructions (pas d'arbre n-aire classique)
- Chaque instruction porte son **numéro de ligne** pour la traçabilité des erreurs
- Les instructions composées (if, repeat) embarquent un pointeur vers un sous-AST (instruction / bloc)
- Fonctions de construction dédiées : ast_decl_registre, ast_porte_1q, ast_if, etc.

AST — exemple : état de Bell



La liste chaînée (flèches en pointillé = champ suivant) reflète l'ordre séquentiel du programme source, tandis que chaque nœud encapsule ses attributs propres via l'union.

Rôle de l'analyse sémantique (semantic.c)

- Parcourt l'AST une seconde fois pour vérifier des propriétés que la grammaire seule ne peut exprimer.
- Maintient une **table des symboles** chaînée (Symbole) : nom, type (QUANTIQUE/CLASSIQUE), taille, ligne de déclaration, et pour les registres classiques un tableau bool* mesure (bit déjà mesuré ou non).
- Comptabilise le nombre total de qubits déclarés (nb_qubits_total) avec avertissement au-delà de MAX_QUBITS = 20 (limite dictée par la taille 2^n de l'espace d'état simulé).

Fonctions clés

trouver_symbole, ajouter_symbole, verifier_reference (bornes d'indice),
marquer_mesure / bit_est_mesure, analyser_noeud (dispatch par TypeNoeud)

Règles de vérification statique

Vérification	Sévérité
Registre redéclaré sous le même nom	Erreur
Registre inconnu / indice hors bornes	Erreur
Porte quantique appliquée à un registre classique	Erreur
Angle manquant pour une porte de rotation	Erreur
Angle fourni pour une porte sans paramètre	Avertissement
Contrôle et cible identiques (cx, cz, Toffoli)	Erreur
Mesure vers un registre non classique / depuis un registre non quantique	Erreur
Condition if portant sur un registre quantique	Erreur
Bit classique testé dans un if sans avoir été mesuré au préalable	Avertissement

Exemple de diagnostic

Erreur sémantique (ligne 7) : Porte quantique sur registre classique 'c'.

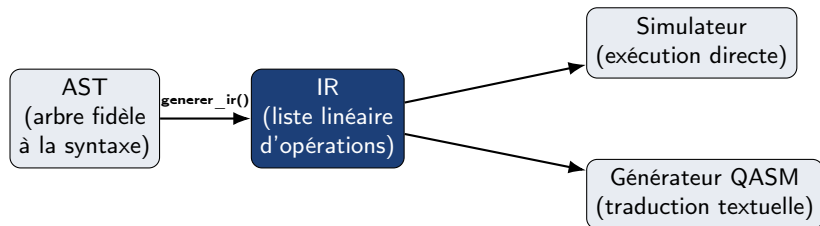
Structure de l'IR — ir.h

```
typedef enum {
    OP_H, OP_X, OP_Y, OP_Z, OP_S, OP_T,
    OP_RX, OP_RY, OP_RZ,
    OP_CX, OP_CZ, OP_SWAP, OP_TOFFOLI,
    OP_MEASURE, OP_IF_DEBUT, OP_IF_FIN,
    OP_BARRIER, OP_PRINT_STATE, OP_PRINT_PROBS
} OpCode;

typedef struct InstructionIR {
    OpCode op;
    int qubits[3];      int nb_qubits;
    int bit_classique;   double angle;
    int condition_bit;   int condition_val;
    int ligne_source;    int registre_id;
    struct InstructionIR *suivant;
} InstructionIR;
```

- L'IR est une **liste linéaire** d'instructions à plat : les qubits multi-registres sont ramenés à des **indices globaux** via une table `RegistreMapping` (nom → qubit de départ, taille).
- Les blocs if sont *aplatis* en une paire de marqueurs `OP_IF_DEBUT` / `OP_IF_FIN` encadrant les instructions conditionnelles — indépendant du back-end consommateur (simulateur ou QASM).

De l'AST à l'IR — indépendance des back-ends



- L'IR découple la *sémantique du langage* de son *exécution* : un même flux d'InstructionIR alimente indifféremment le simulateur ou le générateur de code.
- Ce découplage facilite l'ajout futur d'autres back-ends (ex. transpilation vers un SDK tiers) sans toucher au front-end (lexer/parser/AST).

- Un registre de n qubits est représenté par un vecteur complexe de dimension 2^n :

$$|\psi\rangle = \sum_{i=0}^{2^n-1} \alpha_i |i\rangle, \quad \alpha_i \in \mathbb{C}, \quad \sum_i |\alpha_i|^2 = 1$$

- Implémentation : `double complex*` `etat` de taille 1 « `nb_qubits`, initialisé à $|0\dots 0\rangle$ (`etat[0] = 1.0`).
- Une porte à 1 qubit sur le qubit k agit par **paires d'indices** $(i, i \oplus 2^k)$: c'est le cœur de `sim_appliquer_porte_1q`, qui applique une matrice 2×2 à chaque paire sans reconstruire l'état complet.

Complexité

Chaque porte 1-qubit coûte $O(2^n)$; la limite `MAX_QUBITS=20` borne la mémoire à 2^{20} amplitudes complexes (≈ 16 Mo en `double complex`).

Portes à un qubit — matrices implémentées

$$\begin{aligned} H &= \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} & X &= \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} & Y &= \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix} & R_x(\theta) &= \begin{pmatrix} \cos \frac{\theta}{2} & -i \sin \frac{\theta}{2} \\ -i \sin \frac{\theta}{2} & \cos \frac{\theta}{2} \end{pmatrix} \\ Z &= \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} & S &= \begin{pmatrix} 1 & 0 \\ 0 & i \end{pmatrix} & T &= \begin{pmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{pmatrix} & R_y(\theta) &= \begin{pmatrix} \cos \frac{\theta}{2} & -\sin \frac{\theta}{2} \\ \sin \frac{\theta}{2} & \cos \frac{\theta}{2} \end{pmatrix} \\ & & & & & R_z(\theta) &= \begin{pmatrix} e^{-i\theta/2} & 0 \\ 0 & e^{i\theta/2} \end{pmatrix} \end{aligned}$$

Chaque matrice est construite localement dans `sim_appliquer_ir` (dispatch sur `OpCode`) puis déléguée à la fonction générique `sim_appliquer_porte_1q`.

Portes multi-qubits — implémentation par permutation/phase

- **CX (CNOT)** : pour tout indice i avec le bit de contrôle à 1, échange les amplitudes i et $i \oplus 2^{\text{cible}}$ (`sim_appliquer_cx`).
- **CZ** : applique une phase -1 lorsque les deux bits (contrôle et cible) valent 1 — aucune permutation nécessaire (`sim_appliquer_cz`).
- **SWAP** : échange les amplitudes correspondant à l'inversion des bits q_1 et q_2 (`sim_appliquer_swap`).
- **Toffoli (CCNOT)** : généralisation du CNOT à deux contrôles ; échange $i \leftrightarrow i \oplus 2^{\text{cible}}$ si les deux bits de contrôle valent 1 (`sim_appliquer_toffoli`).

Observation

Toutes ces portes sont **unitaires par construction** car implémentées comme des permutations d'amplitudes (ou des changements de phase de module 1), garantissant la conservation de la norme sans vérification explicite supplémentaire.

- 1 Calcul de $p_0 = \sum_{i:\text{bit}_k(i)=0} |\alpha_i|^2$ (probabilité d'obtenir 0 sur le qubit k).
- 2 Tirage d'un nombre pseudo-aléatoire $r \sim \mathcal{U}(0,1)$ via `rand()` (graine `srand(seed)` pour la reproductibilité, ou `time(NULL)` sinon).
- 3 Résultat = 1 si $r > p_0$, sinon 0 ; le bit classique correspondant est mis à jour.
- 4 **Projection** : toutes les amplitudes incompatibles avec le résultat sont mises à zéro.
- 5 **Renormalisation** : division par $\sqrt{\text{norme restante}}$ pour restaurer $\sum |\alpha_i|^2 = 1$.

Point d'attention identifié

Le test $r > p_0$ pour choisir le résultat 1 est une convention arbitraire mais cohérente ; la reproductibilité totale d'une exécution dépend uniquement de la graine passée en `-seed`.

Structures de contrôle dans l'IR — exemple if

```
case OP_IF_DEBUT: {
    int val = sim->bits_classiques[courant->condition_bit];
    if (val == courant->condition_val) {
        courant = courant->suivant;
        while (courant && courant->op != OP_IF_FIN) {
            sim_appliquer_ir(sim, courant);
            courant = courant->suivant;
        }
    } else {
        while (courant && courant->op != OP_IF_FIN) {
            courant = courant->suivant;
        }
    }
    break;
}
```

Le simulateur interprète directement la paire OP_IF_DEBUT/OP_IF_FIN comme un *saut conditionnel* sur la liste chaînée d'IR — sans backpatching ni structure arborescente, contrairement à l'AST.

Export vers OpenQASM 2.0 (codegen_qasm.c)

Principe

- Un unique parcours de l'IR détermine nb_qubits et nb_bits par balayage des champs qubits[] / bit_classique.
- En-tête standard généré automatiquement :

```
OPENQASM 2.0;  
include "qelib1.inc";  
qreg q[N]; creg c[M];
```

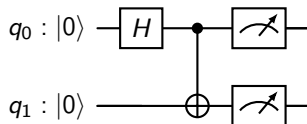
- Traduction directe OpCode → instruction QASM (h, cx, rz(theta), measure ... -> ..., if (c==v) { ... }, barrier, ccx pour Toffoli).
- Les print state/print probs (propres à la simulation) sont convertis en **commentaires** QASM.

```
1  OPENQASM 2.0;  
2  include "qelib1.inc";  
3  
4  qreg q[2];  
5  creg c[2];  
6  
7  h q[0];  
8  cx q[0], q[1];  
9  measure q[0] -> c[0];  
10 measure q[1] -> c[1];  
11 // print probs (simulation uniquement)
```

Sortie réelle pour le programme bell.qtl — directement importable dans Qiskit, IBM Quantum Experience ou tout outil compatible OpenQASM 2.0.

Cas d'étude 1 — État de Bell (intrication à 2 qubits)

```
qreg q[2];  
creg c[2];  
  
h q[0];  
cx q[0], q[1];  
  
measure q[0] -> c[0];  
measure q[1] -> c[1];  
  
print probs;
```

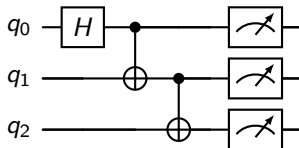


$$|\psi\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$$

Résultats attendus (print probs) :
 $P(00) = P(11) = 0.5$, $P(01) = P(10) = 0$ —
corrélation parfaite entre $c[0]$ et $c[1]$.

Cas d'étude 2 — État GHZ (intrication à 3 qubits)

```
qreg q[3];  
creg c[3];  
  
h q[0];  
cx q[0], q[1];  
cx q[1], q[2];  
  
barrier q[0], q[1], q[2];  
  
measure q[0] -> c[0];  
measure q[1] -> c[1];  
measure q[2] -> c[2];  
  
print probs;
```



$$|\psi\rangle = \frac{1}{\sqrt{2}}(|000\rangle + |111\rangle)$$

La `barrier` illustre la synchronisation explicite avant mesure — sans effet sur l'état simulé mais préservée dans l'export QASM.

Cas d'étude 3 — Algorithme de Deutsch

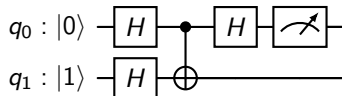
```
qreg q[2];  
creg c[1];
```

```
x q[1];  
h q[0];  
h q[1];
```

```
cx q[0], q[1];
```

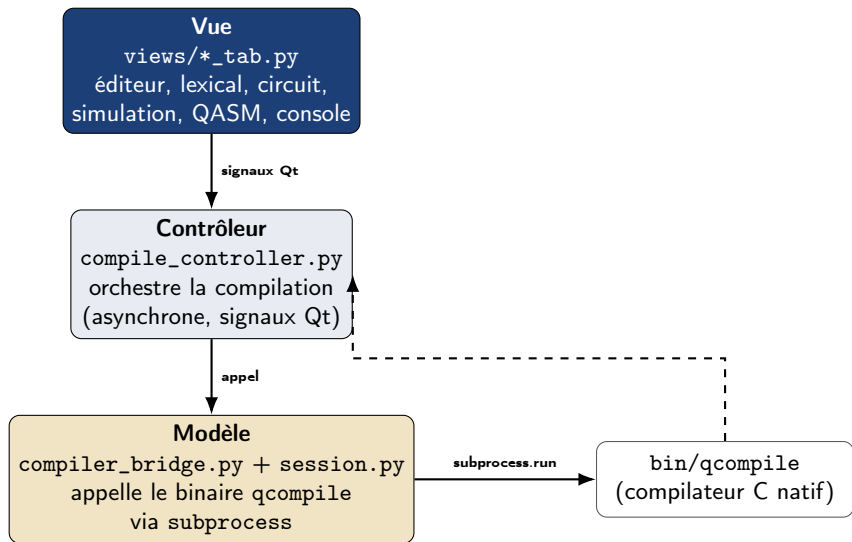
```
h q[0];  
measure q[0] -> c[0];
```

```
if (c[0] == 0) print state;  
if (c[0] == 1) print state;
```



- Illustre l'oracle U_f implémenté ici via cx (fonction équilibrée $f(x) = x$).
- Démontre l'usage du couple `if/print` pour un diagnostic conditionnel post-mesure — test direct des règles sémantiques sur les registres classiques mesurés.

Architecture MVC de la GUI



Le

CompilerBridge localise dynamiquement l'exécutable `qcompile` (plusieurs chemins candidats testés), l'invoque avec un fichier temporaire `.qtl`, puis parse `stdout/stderr`.

Organisation des onglets de résultats

- **Éditeur** — panneau permanent (splitter gauche), coloration syntaxique QuantL
- **Lexical / Syntaxique** — visualisation des tokens et de l'AST
- **Circuit** — représentation graphique du circuit quantique
- **Simulation** — vecteur d'état, probabilités, résultats de mesure
- **Export QASM** — code OpenQASM 2.0 généré, copiable/exportable
- **Console** — sortie brute du compilateur (diagnostics, erreurs)
- Thèmes **clair/sombre** interchangeables (`light.qss` / `dark.qss`)
- Exemples préchargés : État de Bell, GHZ, Deutsch (menu EXAMPLES)

Ergonomie

Contrairement à une interface purement à onglets, l'éditeur reste **toujours visible** à gauche (via un `QSplitter`) : l'utilisateur ne perd jamais le contexte du code source en consultant les résultats.

Chaîne de build (Makefile)

```
quantl.y --(bison -d -v)--> quantl.tab.c / quantl.tab.h
quantl.l --(flex)-----> quantl.yy.c
*.c      --(gcc -std=c11 -Wall -Wextra -O2)--> build/*.o
build/*.o --(gcc, -lm)-----> bin/qcompile
```

- Compilation stricte : `-Wall -Wextra -std=c11`, liaison avec `libm` (fonctions trigonométriques/complexes).
- Cible `make test` exécute automatiquement les trois programmes de référence (`bell.qtl`, `ghz.qtl`, `deutsch.qtl`) avec une graine fixe (`-seed 12345`) pour des résultats reproductibles.
- Cibles `clean` / `distclean` pour la suppression des artefacts générés (`.o`, binaire, fichiers Bison/Flex intermédiaires, `.qasm`).

Tests fonctionnels (tests/)

- `bell.qtl`, `ghz.qtl`, `deutsch.qtl` — circuits de référence complets
- `simple.qtl`, `bell_debug.qtl` — cas minimaux de débogage

Tests structurés par phase

- `lexical_tests/` — validation du tokenizer
- `syntax_tests/` — validation de la grammaire
- `semantic_tests/` — validation des règles statiques

Reproductibilité

L'option `-seed N` garantit qu'une même exécution produit toujours les mêmes résultats de mesure, condition nécessaire pour des tests automatisés déterministes sur un phénomène par nature probabiliste.

Ce qui est fonctionnel de bout en bout

Lexer $\rightarrow$ Parser $\rightarrow$ AST $\rightarrow$ Analyse sémantique $\rightarrow$ IR $\rightarrow$ {Simulateur, Export QASM 2.0}, avec interface graphique complète appelant le compilateur natif.

- Couverture des primitives essentielles du calcul quantique par circuits : portes 1/2/3-qubits, mesure, contrôle conditionnel classique, synchronisation, répétition.
- Simulateur numériquement correct par construction (portes unitaires implémentées comme permutations/phases sur le vecteur d'état).
- Interopérabilité assurée via l'export OpenQASM 2.0, standard reconnu de l'écosystème (Qiskit, IBM Quantum).

- **Simulation idéale uniquement** : aucun modèle de bruit (dépolariation, relaxation T_1/T_2 , portes bruitées) — le simulateur suppose un matériel parfait.
- **Passage à l'échelle** : complexité mémoire en $O(2^n)$, limite pragmatique fixée à 20 qubits (16 Mo d'amplitudes complexes).
- **Pas d'optimisation de circuit** : aucune fusion/réécriture de portes avant simulation ou export (contrairement aux transpileurs de SDK matures).
- `repeat` génère une ré-exécution complète du bloc à chaque simulation, sans réutilisation d'état entre itérations autre que via les registres classiques.
- Le fichier `json_writer.c/.h` est présent mais vide — fonctionnalité d'export JSON planifiée mais non implémentée.

Extensions du langage

- Fonctions/sous-circuits réutilisables (modularité)
- Boucles paramétrées par variable (au-delà de `repeat(N)` constant)
- Types de données classiques plus riches (entiers signés, tableaux)

Modèle physique

- Modélisation du bruit et de la décohérence
- Bascule vers un back-end **Qiskit** réel pour exécution sur matériel/simulateurs cloud

Outillage

- Optimiseur de circuits (fusion de portes, réduction de profondeur)
- Export JSON structuré (finaliser `json_writer`)
- Visualisation graphique interactive du circuit dans l'onglet *Circuit*
- Tests automatisés end-to-end intégrés (CI)

Synthèse

QuantL démontre qu'il est possible de bâtir, avec des outils de compilation classiques (**Flex**, **Bison**, **C**), un langage dédié à la programmation de circuits quantiques, complet depuis l'analyse lexicale jusqu'à la simulation numérique et l'interopérabilité via OpenQASM, le tout encapsulé dans une interface graphique accessible (**PyQt6**).

Merci de votre attention

Questions & discussion